Software Project Management

Estimating Project Size And Complexity

Estimations

- Very difficult to do, but needed often
- Created, used or refined during
  - Strategic planning
  - Feasibility study and/or SOW
  - Proposals
  - Vendor and sub-contractor evaluation
  - Project planning (iteratively)
- Basic process
  1) Estimate the **size** of the product
  2) Estimate the **effort** (person-months)
  3) Estimate the **schedule**
  - NOTE: Not all of these steps are always explicitly performed

Estimations

- Remember, an “exact estimate” is an oxymoron
- Estimate how long will it take you to get home from class tonight
  - On what basis did you do that?
  - Experience right?
  - Likely as an “average” probability
  - For most software projects there is no such ‘average’
- Most software estimations are off by 25-100%

Estimation

- Target vs. Committed Dates
  - Target: Proposed by business or marketing
    - » Do not commit to this too soon!
  - Committed: Team agrees to this
    - » After you've developed a schedule
- Size:
  - Small projects (10-99 FPs), variance of 7% from post-requirements estimates
  - Medium (100-999 FPs), 22% variance
  - Large (1000-9999 FPs) 38% variance
  - Very large (> 10K FPs) 51% variance

Estimation Methodologies

- Top-down
- Bottom-up
- Analogy
- Expert Judgment
- Priced to Win
- Parametric or Algorithmic Method
  - Using formulas and equations

Top-down Estimation

- Based on overall characteristics of project
  - Some of the others can be “types” of top-down (Analogy, Expert Judgment, and Algorithmic methods)
- Advantages
  - Easy to calculate
  - Effective early on (like initial cost estimates)
- Disadvantages
  - Some models are questionable or may not fit
  - Less accurate because it doesn't look at details

Bottom-up Estimation

- Create WBS
- Add from the bottom-up
- Advantages
  - Works well if activities well understood
- Disadvantages
  - Specific activities not always known
  - More time consuming

Expert Judgment

- Use somebody who has recent experience on a similar project
- You get a “guesstimate”
- Accuracy depends on their ‘real’ expertise
- Comparable application(s) must be accurately chosen
  - Systematic
- Can use a weighted-average of opinions

Estimation by Analogy

- Use past project
  - Must be sufficiently similar (technology, type, organization)
  - Find comparable attributes (ex: # of inputs/outputs)
  - Can create a function
- Advantages
  - Based on actual historical data
- Disadvantages
  - Difficulty ‘matching’ project types
  - Prior data may have been mis-measured
  - How to measure differences – no two exactly same

Priced to Win

- Just follow other estimates
- Save on doing full estimate
- Needs information on other estimates (or prices)
- Purchaser must closely watch trade-offs
- Priced to lose?

Algorithmic Measures

- Lines of Code (LOC)
- Function points
  - Feature points or object points
- Other possible
  - Number of bubbles (processes) on a DFD (Data Flow Diagram)
  - Number of ERD (Entity-Relationship Diagram) entities
  - Number of processes on a structure chart
- LOC and function points most common (of the algorithmic approaches)
- Majority of projects use none of the above

Code-based Estimates

- LOC Advantages
  - Commonly understood metric
  - Permits specific comparison
  - Actually it is easily measured
- LOC Disadvantages
  - Difficult to estimate early in cycle
  - Counts vary by language
  - Many costs not considered (ex: requirements)
  - Programmers may be rewarded based on this
    - Can use: # defects/# LOC
  - Code generators produce excess code

LOC Estimate Issues

- How do you know how many in advance?
- What about different languages?
- What about programmer style?
- Stat: avg. programmer productivity: 3,000 LOC/yr
- Most algorithmic approaches are more effective after requirements (or have to be after)

Function Points

- Software size should be measured by number & complexity of functions it performs
- More methodical than LOC counts
- House analogy
  - House's Square Feet ~= Software LOC
  - # Bedrooms & Baths ~= Function points
  - Former is size only, latter is size & function

Function Point Process

Four basic steps:

1. Count # of business functions per category
   - Categories: outputs, inputs, db inquiries, files or data structures, and interfaces
2. Establish Complexity Factor for each and apply
   - Simple, Average, Complex
   - Set a weighting multiplier for each (0 –>15)
   - This results in the "unadjusted function-point total"
3. Compute an "influence multiplier" and apply
   - It ranges from 0.65 to 1.35; is based on 14 factors
4. Results in "function point total"
   - This can be used in comparative estimates

Function Point Analysis

- The influence multiplier (a.k.a. *the Value Adjustment Factor* or *Technical Complexity Factor*) is a metric used in Function Point Analysis (FPA) to refine a project's size estimate based on its technical complexity.
- It scales the initial "unadjusted" count by a factor ranging from 0.65 to 1.35.
- **Identify the 14 Influence Factors:**
  - The multiplier is derived from 14 **General System Characteristics** (GSCs). Each factor is rated on a scale of **0 (no influence)** to **5 (strong influence)**:

14 General System Characteristics

1. **Data communications:** How much data is sent/received over communication lines?
2. **Distributed data processing:** Does the system use distributed databases or processing?
3. **Performance:** Are there strict response time or throughput requirements?
4. **Heavily used configuration:** Will the system run on a heavily used operational environment?
5. **Transaction rate:** Is the transaction rate high (peak periods)?
6. **Online data entry:** What percentage of data is entered online?
7. **End-user efficiency:** Is the application designed for maximum user ease?
8. **Online update:** Are logical internal files updated online?
9. **Complex processing:** Does the system involve complex logic or math?
10. **Reusability:** Is the code designed to be reused in other applications?
11. **Installation ease:** Are conversion and installation automated?
12. **Operational ease:** Are start-up, back-up, and recovery automated?
13. **Multiple sites:** Is the system designed for multiple organizations/sites?
14. **Facilitate change:** Is the system designed for easy modification?

Function Point Calculation

Calculate the Total Degree of Influence (TDI):

- Sum the ratings (r_i) of all 14 factors to find the Total Degree of Influence (TDI):

$$TDI = \sum_{i=1}^{14} r_i$$

- Since each factor ranges from 0 to 5, the TDI can range from 0 to 70 (14 x 5 = 70)

Compute the Influence Multiplier (VAF)

- The Value Adjustment Factor (VAF) is calculated using the following formula:

$$\text{VAF} = 0.65 + (0.01 \times \text{TDI})$$

- Minimum: If TDI = 0, then VAF = 0.65 + (0.01 × 0) = $\mathbf{0.65}$ (reduces size by 35%).
- Maximum: If TDI = 70, then VAF = 0.65 + (0.01 × 70) = $\mathbf{1.35}$ (increases size by 35%).

Function Point Calculation

Apply the Multiplier

- The final Function Point (FP) count is obtained by multiplying the Unadjusted Function Points (UFP)—which are based on inputs, outputs, files, etc.—by the VAF:

$$FP = UFP \times VAF$$

Example:

- If a project has a UFP of 100 and a TDI of 35 (average complexity):

- 1. $VAF = 0.65 + (0.01 \times 35) = 1.00$
- 2. $FP = 100 \times 1.00 = \mathbf{100}$ Function Points

Wideband Delphi

- Group consensus approach
- Rand corp. used orig. Delphi approach to predict future technologies
- Present experts with a problem and response form
- Conduct group discussion, collect anonymous opinions, then feedback
- Conduct another discussion & iterate until consensus
- Advantages
  - Easy, inexpensive, utilizes expertise of several people
  - Does not require historical data
- Disadvantages
  - Difficult to repeat
  - May fail to reach consensus, reach wrong one, or all may have same bias

Effort Estimation

- Now that you know the “size”, determine the “effort” needed to build it
- Various models: empirical, mathematical, subjective
- Expressed in units of duration
  - person-months (or ‘staff-months’ now)
- Efficiency/productivity of people
  - Motivation
  - Ability of team
  - Application experience
  - Range of personnel

Duration

- What is the duration?
  The duration of a project is the elapsed time in business working days, not including weekends, holidays, or other nonworking days.
- Duration is different from work effort.
  - Work effort is labor required to complete an activity. That labor can be consecutive or nonconsecutive hours.
  - Duration is work effort divided by number of people doing the work:
  - Or, work effort is the duration multiplied by the number of people doing the work
  - E.g. five people reviewing a document; the duration may be 10 hours but the work effort is 50 staff-hours

Duration

- When we talk about estimates and duration, there are two types of time that are not the same:
  - Labor Time
  - Clock time
- For example, if we estimate that a task would require 40 hours of labor to complete, then in a normal business setting we need at least 50 business hours, ***Why***?

Duration

- Your average business day may include few of the following activities or perhaps a number of each:
  - Meetings – non-project related
  - Phone calls
  - E-mails
  - Coffee breaks
  - Lunches
  - Friendly chat with your teammates
- The duration of the activity is the clock time or “Elapsed time” that we want to estimate.
- Productivity is usually 50-75% of a business day

Variation in Activity Duration

- Since we cannot know what factors will be effective when work is underway on an activity, we cannot know exactly how long it will take.
- One of the goals in estimating the activity duration is to define the activity to a level of granularity that estimates have a narrow variance; the estimate is as good as you can get it at the planning stages of the project.
- There are several causes of variation in the actual activity duration:
  - Varying skill levels
  - Unexpected events
  - Efficiency of work time
  - Mistakes and Misunderstandings

Causes of Variation

Causes of Variation in the Actual Activity Duration

- **Varying skill levels.** Our estimate of the activity duration is based on average skilled engineer. We may get a higher or lower skilled engineer assigned to the activity, causing the actual duration to vary from the planned duration.
- **Unexpected events.** Random acts of nature, vendor delays, traffic jams, power failures, etc.
- **Efficiency of work time.** Every time a worker is interrupted it takes more time to get up to the level of productivity prior to the time of the interruption. These interruptions may have little or substantial impact on the worker's productivity
- **Mistakes and Misunderstandings.** In organizations that have a documented process, rework may have its toll on the actual activity duration.

Estimation Issues

- Quality estimations needed early but information is limited
- Precise estimation data available at end but not needed
  - Or is it? What about the next project?
  - Should have determined just how accurate your original estimate was.
- Best estimates are based on past experience
- Politics of estimation:
  - You may anticipate a “cut” by upper management
- For many software projects there is little or no data for estimation
  - Technologies change
  - Historical data unavailable
  - Wide variance in project experiences/types
  - Subjective nature of software estimation

Over and Under Estimation

- Over estimation issues
  - The project will not be funded
    - Conservative estimates guaranteeing 100% success may mean funding probability of zero.
  - Parkinson's Law: Work expands to take the time allowed
  - Danger of feature and scope creep
  - Be aware of “double-padding”: team member + manager
- Under estimation issues
  - Quality issues (short changing key phases like testing)
  - Inability to meet deadlines
  - Morale and other team motivation issues

Estimation Guidelines

- Estimate iteratively!
  - Process of gradual refinement
  - Make your best estimates at each planning stage
  - Refine estimates and adjust plans iteratively
  - Plans and decisions can be refined in response
  - Balance: too many revisions vs. too few

Know Your Deadlines

- Are they ‘Real Deadlines’?
  - Tied to an external event
  - Have to be met for project to be a success
  - Ex: end of financial year, contractual deadline, Y2K
- Or ‘Artificial Deadlines’?
  - Set by arbitrary authority
  - May have some flexibility (if pushed)

Estimation "Presentation"

- How you present the estimation can have huge impact
- Techniques
  - Plus-or-minus qualifiers [6 months ±1 month]
  - Ranges [6-8 months]
  - Risk Quantification
    - ± with added information
      - +1 month if new tools not working as expected
      - -2 weeks for less delay in hiring new developers
  - Cases [Best / Planned / Current / Worst cases]
  - Coarse Dates [Q4 25]
  - Confidence Factors [June 1 – 10% probability, July 1 – 50%, etc.]